

MACHINE LEARNING BOOTCAMP · DAY 3 OF 5



More Models & Unsupervised Learning

Fair testing, stronger models, and learning without labels

Soyinka Enoch Oluwatimilehin · 30/09/2026

Quick recap of Day 2

1

EDA → clean → features
→ split → train →
evaluate

2

Beat a **baseline**: doing
nothing scored 61.5%
on Titanic

3

Overfitting: a tree scored
98% on training, 80%
on test

Warm-up: 99% on training, 70% on test. What's wrong?

Today's plan — 3 hours

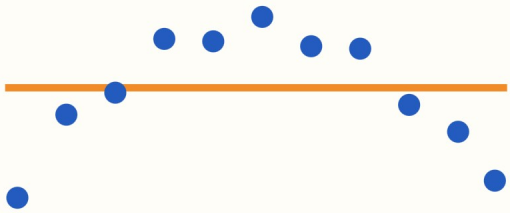
Time	Block	What we'll do
0:00 – 0:10	Welcome	Day 2 recap, today's plan
0:10 – 0:45	Fair testing	Overfitting, cross-validation + Notebook 1
0:45 – 1:25	Stronger models	Forests, boosting, SVM, tuning + Notebook 1
1:25 – 1:35	Break	Stretch, questions
1:35 – 2:25	Unsupervised	Clustering, PCA, anomalies + Notebook 2
2:25 – 2:50	Semi-supervised	Learning from few labels + Notebook 3
2:50 – 3:00	Wrap-up	Choosing a model, recap

PART 1

Testing models fairly

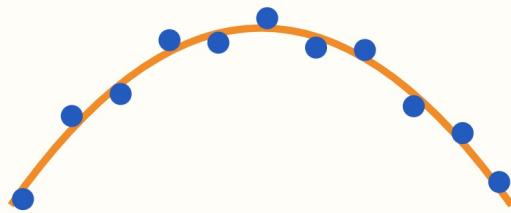
Is the model really good — or just lucky?

Underfitting vs overfitting



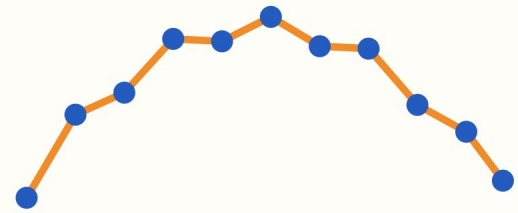
Underfitting

Too simple. Misses the pattern.
Bad on train **and** test.



Just right

Learns the real pattern. Good
on train and test.

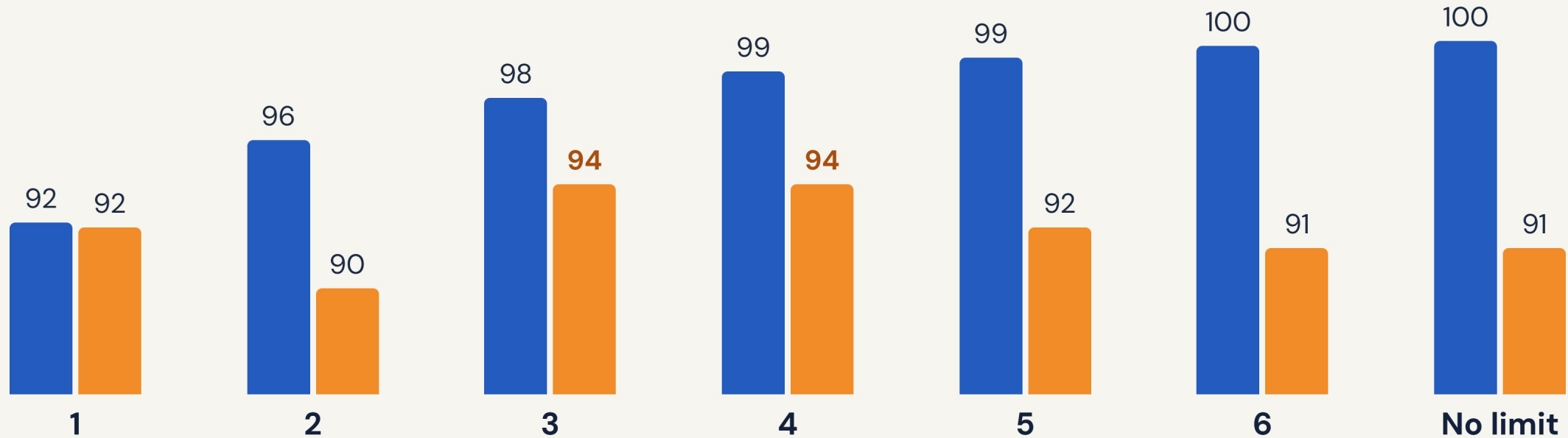


Overfitting

Memorises the noise. Great on
train, worse on test.

Tree depth vs overfitting

■ Training ■ Test Breast cancer data · bars start at 85%



Max tree depth → **Best test score at depth 3–4 (93.9%)**. Deeper = memorising.

One test split can be lucky

87.7% → 96.5%

Same tree, same data — only the random split changed

Split 1: 92.1%	Split 2: 95.6%	Split 3: 95.6%	Split 4: 93.9%	Split 5: 87.7%
Split 6: 93.0%	Split 7: 96.5%	Split 8: 93.0%	Split 9: 87.7%	Split 10: 93.9%

So which number do we report?

Cross-validation: 5 exams, not 1

Round 1	test					96.7%
Round 2		test				98.9%
Round 3			test			97.8%
Round 4				test		98.9%
Round 5					test	96.7%

Report the average: $97.8\% \pm 1.0\%$

```
cross_val_score(model, X, y, cv=5)
```

</> HANDS-ON · 15 MINUTES

Notebook 1 · Parts 1–2

Day3_1_Model_Showdown.ipynb

Part 1 · Meet the breast cancer data

Part 2 · Draw the overfitting curve, test lucky splits, run cross-validation

PART 2

Stronger models

Forests, boosting and SVMs — and when simple wins.

Ensembles: wisdom of the crowd

Many average models voting together usually beat one expert.

BAGGING · SIDE BY SIDE

Random forest

Many trees, each trained on a random sample of the data. They vote.

BOOSTING · ONE AFTER ANOTHER

Gradient boosting

Trees built in sequence. Each one fixes the last one's mistakes.

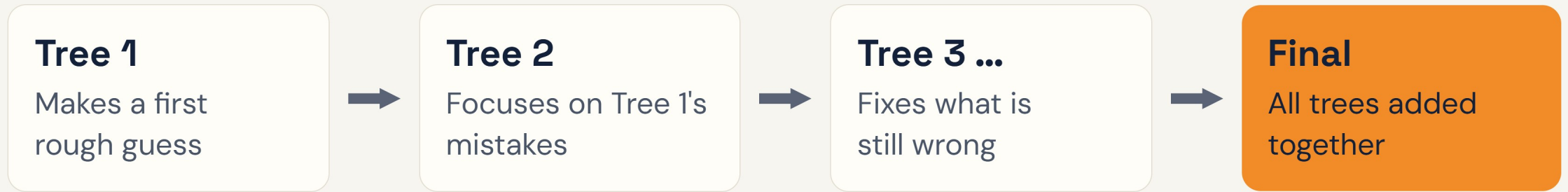
Random forest



- Each tree sees a **random sample** of rows and features
- Their mistakes differ, so they **cancel out**
- Little tuning, **no scaling** needed
- Shows which **features matter most**

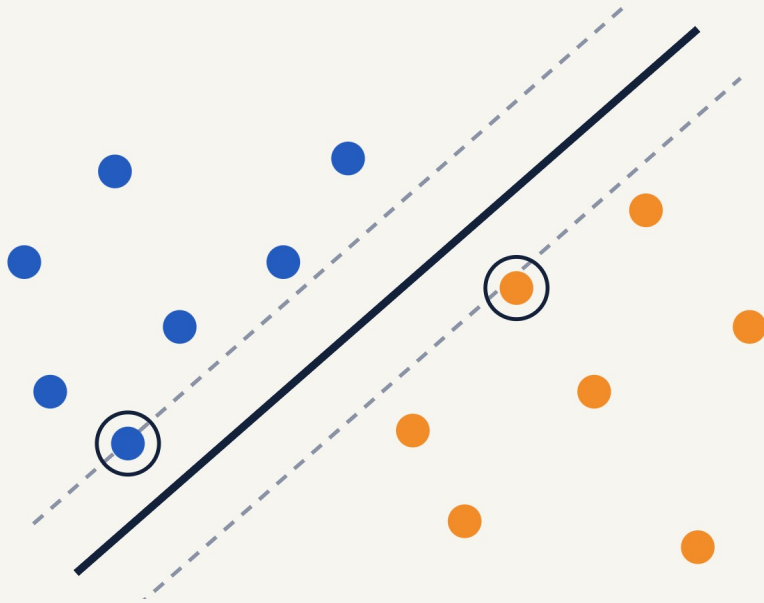
Top features for breast cancer: worst perimeter, worst area, worst concave points

Boosting: learn from mistakes



- Often the **top performer on tables of data**
- Popular libraries: **XGBoost, LightGBM, CatBoost**
- Needs more careful tuning than a random forest

Support Vector Machine (SVM)



Finds the boundary with the **widest safety margin**

The circled points on the margin are the **support vectors**

Kernel trick: can draw curved boundaries too (RBF kernel)

Needs scaling · great on small and medium datasets

Model showdown: breast cancer

569 tumours · 30 measurements · malignant or benign · 5-fold cross-validation

Model	CV accuracy	Spread	Final test
Logistic regression	97.8%	± 1.0	98.2%
SVM (RBF)	96.9%	± 1.5	98.2%
Random forest	96.3%	± 1.8	95.6%
KNN (k = 5)	96.3%	± 1.1	95.6%
Gradient boosting	95.2%	± 1.5	95.6%
Decision tree	91.6%	± 1.8	91.2%

The simplest model won here. Always try simple first.

Tuning: find the best settings

$\gamma \setminus C$	0.1	1	10	100
0.001	73.4	94.5	97.1	97.1
0.01	94.7	97.1	97.6	96.5
0.1	93.8	95.6	94.1	94.3
"scale"	94.3	96.9	96.9	96.7

Hyperparameters are settings you choose before training (depth, k, C, γ)

GridSearchCV tries every combination with cross-validation

Best: C = 10, γ = 0.01 → **97.6%**

Pipelines: prep + model in one

```
num = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())
cat = make_pipeline(SimpleImputer(strategy="most_frequent"), OneHotEncoder())
prep = ColumnTransformer([("num", num, num_cols), ("cat", cat, cat_cols)])

model = make_pipeline(prepare, GradientBoostingClassifier())
cross_val_score(model, X, y, cv=5)
```

Titanic, done properly (5-fold cross-validation):

Logistic **79.4%**

SVM **82.6%**

Forest **82.7%**

Boosting **83.2%**

No leakage: the median is learned **inside each training fold only**.

</> HANDS-ON · 13 MINUTES

Notebook 1 · Parts 3–5

Day3_1_Model_Showdown.ipynb

Part 3 · The model showdown

Part 4 · Tune an SVM with GridSearchCV

Part 5 · Titanic done properly with a pipeline

PART 3

Unsupervised learning

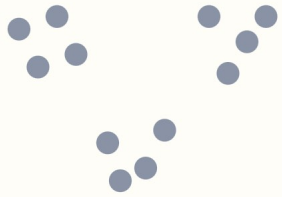
No labels: find groups, simplify data, spot the odd one out.

K-Means: finding groups

1

Choose k

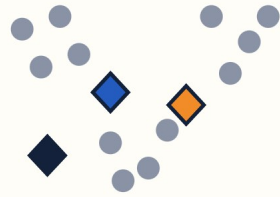
How many groups? Here
 $k = 3$



2

Drop centres

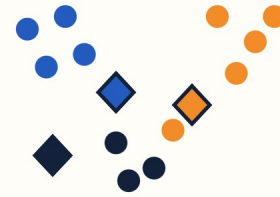
Place k centres
at random



3

Assign

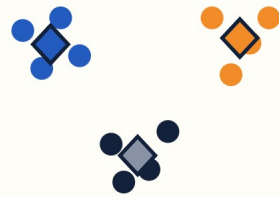
Each point joins its
nearest centre



4

Move & repeat

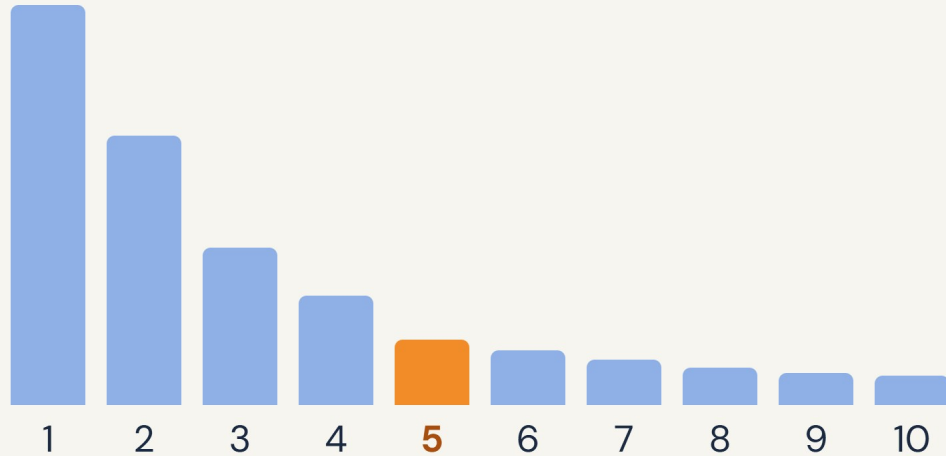
Centres move to
the middle. Repeat
until stable.



```
KMeans(n_clusters=5).fit(X) # no y – no labels!
```

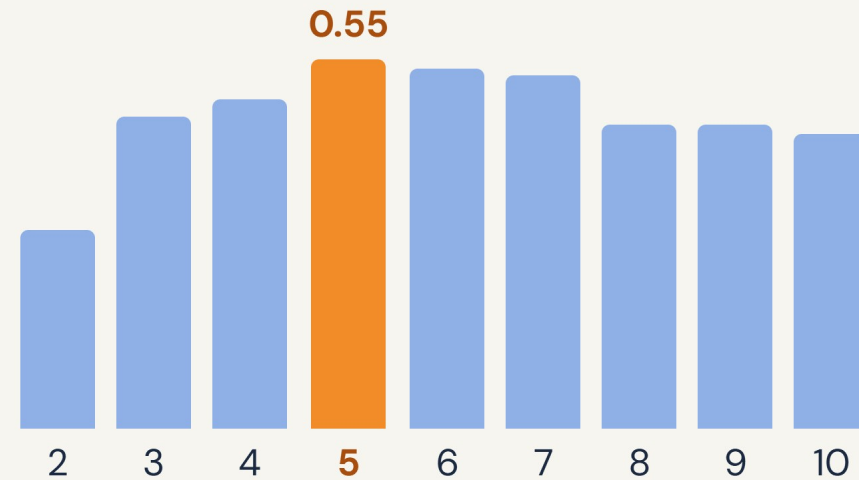
How many groups? Pick k

Elbow method



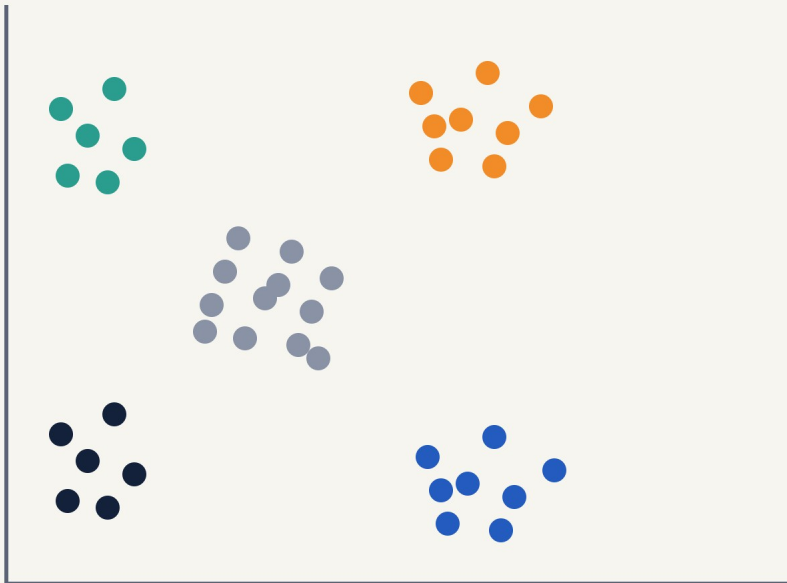
Spread inside groups. It stops falling fast after $k = 5$ — the "elbow".

Silhouette score



How well separated the groups are. Highest at $k = 5$ (0.55).

5 customer groups, no labels



Income → · Spending score ↑ · 200 customers

Target customers · 39
High income, high spending · age ~33

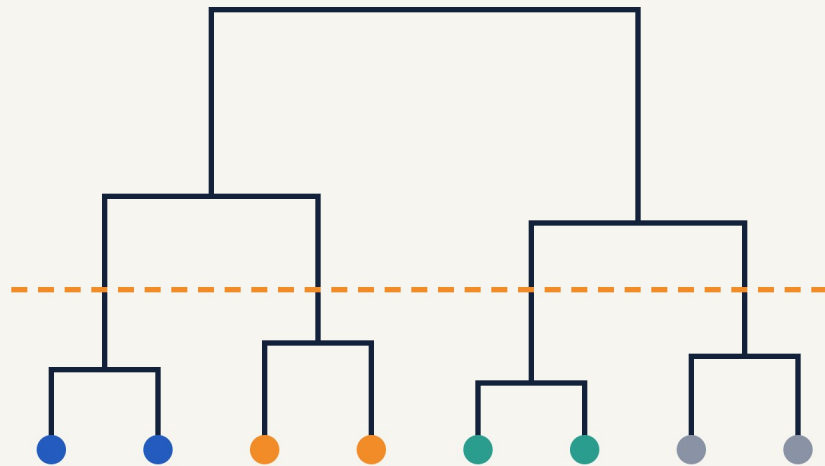
Careful earners · 35
High income, low spending · age ~41

Young impulse buyers · 22
Low income, high spending · age ~25

Budget shoppers · 23
Low income, low spending · age ~45

Average customers · 81
Middle income and spending · age ~43

Hierarchical clustering



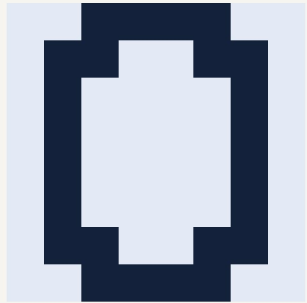
Start: **every point is its own group**

Repeatedly **merge the two closest groups**

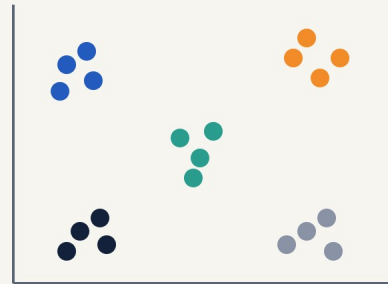
Cut the tree at any height to choose the number of groups

Mall data: almost the same 5 groups as K-Means (agreement 0.94 of 1)

PCA: many columns into a few



64 pixel columns



2 new columns

Finds new directions that keep the **most information**

Never looks at the labels

Plot high-dimensional data

Speed up slow models

Remove noise

PCA on handwritten digits

28.5%

of the information kept by just
2 columns

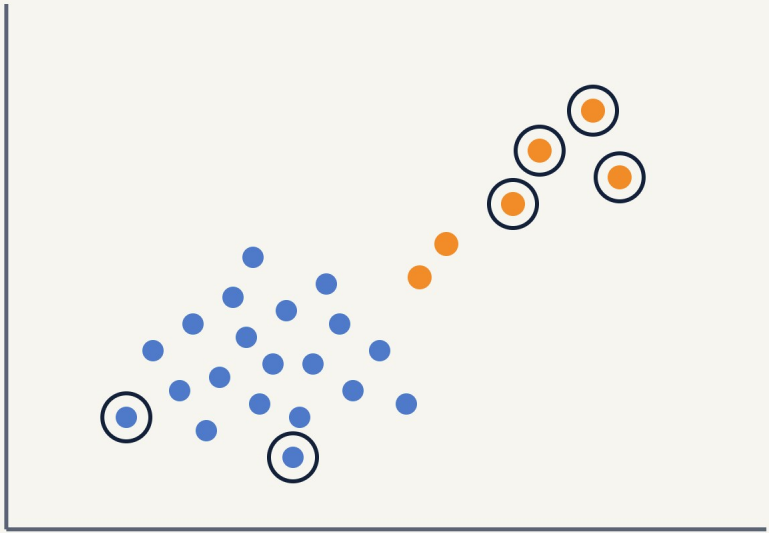
21

columns keep 90% of
the information

Columns kept	Accuracy
2	52.5%
10	87.8%
20	94.2%
All 64	97.2%

20 instead of 64 columns: 3× smaller, only 3 points less accurate.

Anomaly detection



Temperature → · Vibration ↑ · circled = flagged

500 motor readings · 20 from a **faulty motor**

13/20
faults caught

7
false alarms

Isolation Forest: odd points are easy to cut off with a few random splits

Also used for: fraud, abnormal ECG, network attacks

</> HANDS-ON · 20 MINUTES

Notebook 2 · all parts

Day3_2_Unsupervised.ipynb

Parts 1–3 · Group the mall customers (K-Means, hierarchical)

Part 4 · Squeeze digit images with PCA

Part 5 · Catch a faulty motor

PART 4

Semi-supervised learning

What if only a few examples have labels?

Labels are expensive

Medical scans

A specialist must look at every single one

Speech

Hours of typing out what was said

Crop photos

An expert must name each disease

50 /
1,437

Our experiment: only **3.5%** of the digit images keep their label. The rest are unlabelled.

Two ways to use unlabelled data

Self-training

1. Train on the few labels
2. Predict the unlabelled data
3. Keep only the **confident** guesses (90%+ sure) as new labels
4. Retrain and repeat

Label spreading

Connect every image to its most similar neighbours

Labels **spread** from labelled images to their neighbours

Like news spreading through a group of friends

Results: 50 labels vs 1,437



With 3.5% of the labels, label spreading got within 2.5 points of the full dataset.

</> HANDS-ON · 14 MINUTES

Notebook 3 · all parts

Day3_3_Semi_Supervised.ipynb

Part 1 · Hide 96.5% of the labels

Parts 2–3 · Self-training and label spreading

Part 4 · How many labels do you really need?

Which model should I try?

Situation	Try first
Predict a number	Linear regression → random forest
Predict a category, easy to explain	Logistic regression, decision tree
Table of data, best accuracy	Gradient boosting, random forest
Small data, clear boundary	SVM (scale first!)
No labels, want groups	K-Means, hierarchical
Too many columns	PCA
Find rare odd cases	Isolation Forest
Few labels, lots of data	Label spreading, self-training
Images, sound, text	Neural networks → tomorrow!

Recap: what we covered today

- 1 Overfitting = memorising; watch the train/test gap
- 2 Cross-validation gives a fair score, not a lucky one
- 3 Forests, boosting and SVMs are strong — but try simple first
- 4 Pipelines stop leakage; GridSearchCV tunes settings
- 5 No labels: K-Means, PCA, anomalies · Few labels go far

BEFORE DAY 4

- Finish the 3 Day 3 notebooks
- K-Means with Age added — do the groups change?
- Tune a random forest with GridSearchCV

Tomorrow: neural networks, deep learning and computer vision

Questions?

Tomorrow: neural networks, deep learning & computer vision.

Soyinka Enoch Oluwatimilehin · 08144801748